백트래킹 기법

하나 골라 보고, 안 맞으면 돌아와 다른 것을 골라 봅니다.

먼저 작은 예로 살펴봅니다

백트래킹

하나 골라 보고, 안 맞으면 돌아와 다른 것을 골라 봅니다.

예: 삼각형 색칠

빨강을 넣어 봤는데 이웃도 빨강이면 지우고 파랑을 시도합니다.

7-1 카드를 순서 있게 고르는 경우의 수

구할 것

서로 다른 카드 중 몇 장을 골라 줄 세우는 경우의 수를 셉니다.

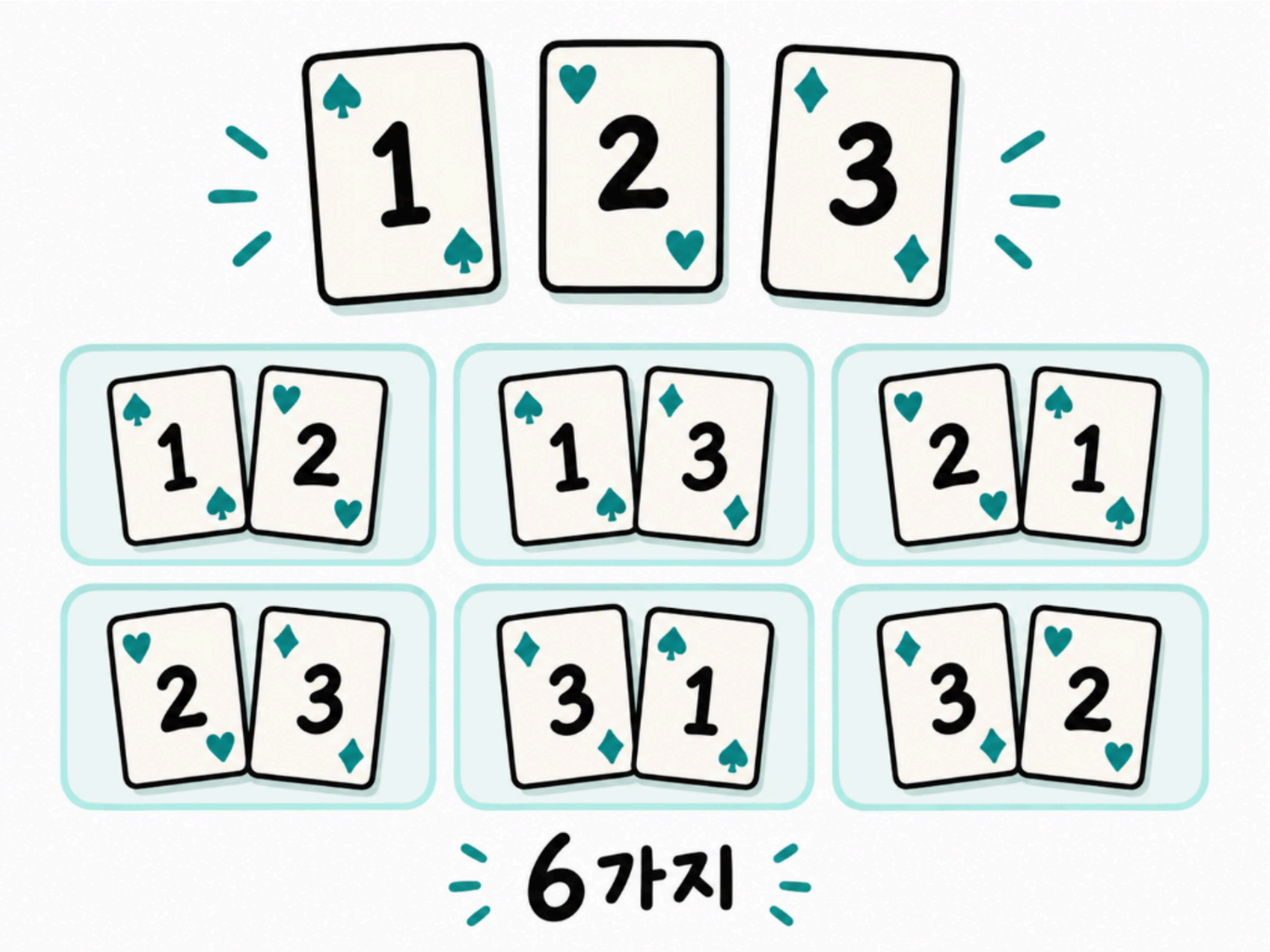
작은 예

카드가 1, 2, 3이고 두 장을 놓습니다.
[1,2]와 [2,1]은 순서가 다르므로 서로 다른 경우입니다.

반환할 값

[1,2], [1,3], [2,1], [2,3], [3,1], [3,2]의 6가지를 반환합니다.

7-1 그림으로 보는 예



첫 자리는 세 가지
둘째 자리는 두 가지
같은 카드는 한 번만
3×2=6가지입니다.

7-1 하나씩 계산한 결과

첫 카드	둘째 카드 후보	경우 수
1	2 또는 3	2
2	1 또는 3	2
3	1 또는 2	2
합계		6

한 장 놓을 때마다 남은 카드 수가 하나 줄어듭니다.

7-1 계산 순서

첫 자리

n장 중 하나를 고를 수 있습니다.

둘째 자리

이미 쓴 한 장을 빼고 n−1장 중에서 고릅니다.

r자리

n×(n−1)×…를 r번 곱합니다.

7-1 변수와 코드의 뜻

변수	n=3, r=2의 변화
answer=1	곱셈을 시작하는 값
i=0	3–0=3을 곱해 answer=3
i=1	3–1=2를 곱해 answer=6
return answer	경우의 수 6 반환

이 문제는 선택 수를 곱하면 됩니다. 다음 문제부터 선택을 직접 나눠 봅니다.

7-1 함수에 들어오는 값

함수

permutation_count(n, r)

입력 조건

0≤r≤n≤8입니다. 같은 카드를 두 번 쓰지 않습니다.

7-1 Python

```
def permutation_count(n, r):
    answer = 1
    for i in range(r):
        answer *= n-i
    return answer
```

시간 O(r), 추가 공간 O(1)

7-1 C

```
int permutation_count(int n, int r) {
    int answer=1;
    for (int i=0; i<r; ++i) answer*=n-i;
    return answer;
}
```

시간 O(r), 추가 공간 O(1)

7-1 다른 입력도 확인합니다

n, r	답	이유
3, 2	6	3×2
3, 3	6	3×2×1
3, 0	1	아무것도 놓지 않는 한 가지
0, 0	1	빈 줄 한 가지

이 문제는 선택 수를 곱하면 됩니다. 다음 문제부터 선택을 직접 나눠 봅니다.

7-2 합이 목표인 부분집합의 개수

구할 것

몇 개의 수를 골라 합이 목표와 같아지는 경우를 셉니다.

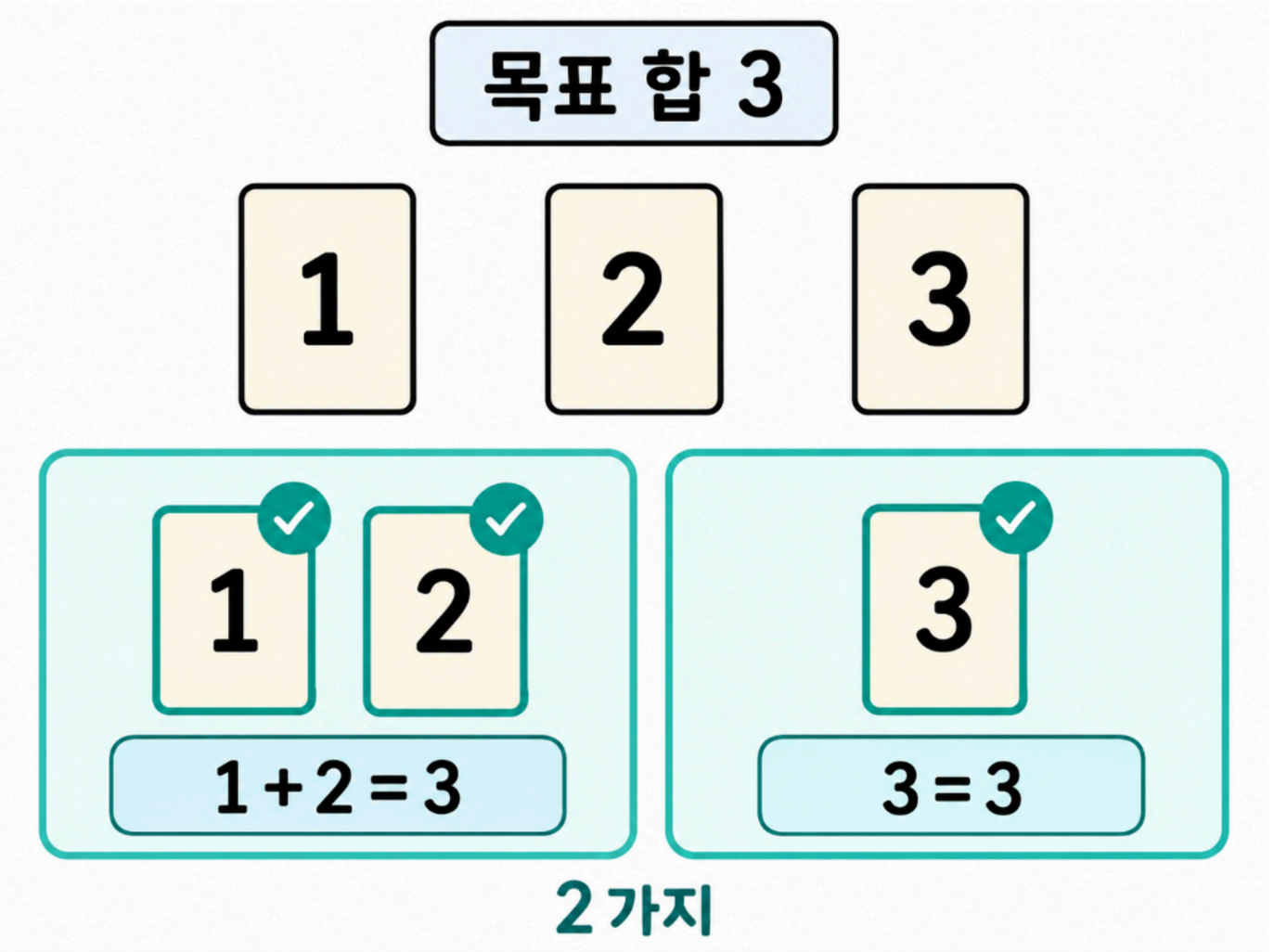
작은 예

a=[1,2,3], target=3입니다.
1과 2를 고르는 방법, 3만 고르는 방법이 있습니다.

반환할 값

가능한 선택이 두 개이므로 2를 반환합니다.

7-2 그림으로 보는 예



1+2=3
3 하나도 합은 3
서로 다른 두 선택
답은 2입니다.

7-2 하나씩 계산한 결과

고른 수	합	목표 3과 같나
없음	0	아니요
1	1	아니요
2	2	아니요
3	3	예
1,2	3	예
1,3 또는 2,3 또는 전부	4, 5, 6	아니요

각 원소에서 고르거나 건너뛰는 두 경우를 모두 봅니다.

search는 남은 선택을 계산하는 함수입니다

search(i,total)은 i번부터 볼 차례이고 지금까지 합이 total이라는 뜻입니다.

호출	지금까지의 선택	다음에 할 일
search(0,0)	아직 안 고름	첫 원소 1을 볼 차례
search(1,0)	1을 건너뜀	다음 원소 2를 볼 차례
search(1,1)	1을 고름	다음 원소 2를 볼 차례

함수 안에서 자기 함수를 다시 호출하는 것을 재귀라고 부릅니다.

7-2 계산 순서

현재 원소를 건너뜀

합은 그대로 두고 다음 원소로 갑니다.

현재 원소를 고름

현재 합에 이 수를 더하고 다음 원소로 갑니다.

끝까지 봤을 때

합이 target이면 1, 아니면 0을 돌려주고 두 갈래를 더합니다.

7-2 변수와 코드의 뜻

변수	뜻
i	지금 볼 원소 번호. 0이면 첫 원소
total	지금까지 고른 수들의 합
search(i+1,total)	이번 수를 건너뛴 갈래
search(i+1,total+a[i])	이번 수를 고른 갈래

백트래킹은 선택을 나눠 살펴보고, 돌아와 다른 선택도 보는 방법입니다.

7-2 함수에 들어오는 값

함수

subset_sum_count(a, target)

입력 조건

원소 0~20개, 각 값 1~50, 목표 0~1000. 같은 값도 위치가 다르면 다른 원소입니다.

7-2 Python

```
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

7-2 C

```
static int subset_search(const int a[], int n, int target,
    int i, int total) {
    if (total>target) return 0;
    if (i==n) return total==target;
    return subset_search(a,n,target,i+1,total)
        + subset_search(a,n,target,i+1,total+a[i]);
}
int subset_sum_count(const int a[], int n, int target) {
    return subset_search(a,n,target,0,0);
}
```

최악 시간 $O(2^n)$, 재귀 스택 $O(n)$

7-2 다른 입력도 확인합니다

입력	답	이유
[], 목표 0	1	아무것도 안 고르는 방법
[], 목표 3	0	합 3을 만들 수 없음
[1,1], 목표 1	2	첫 1과 둘째 1은 다른 원소
[1,2,3], 목표 3	2	1+2 또는 3

백트래킹은 선택을 나눠 살펴보고, 돌아와 다른 선택도 보는 방법입니다.

7-3 이웃한 정점을 다른 색으로 칠하기

구할 것

선으로 연결된 두 점의 색을 다르게 칠할 수 있는지 확인합니다.

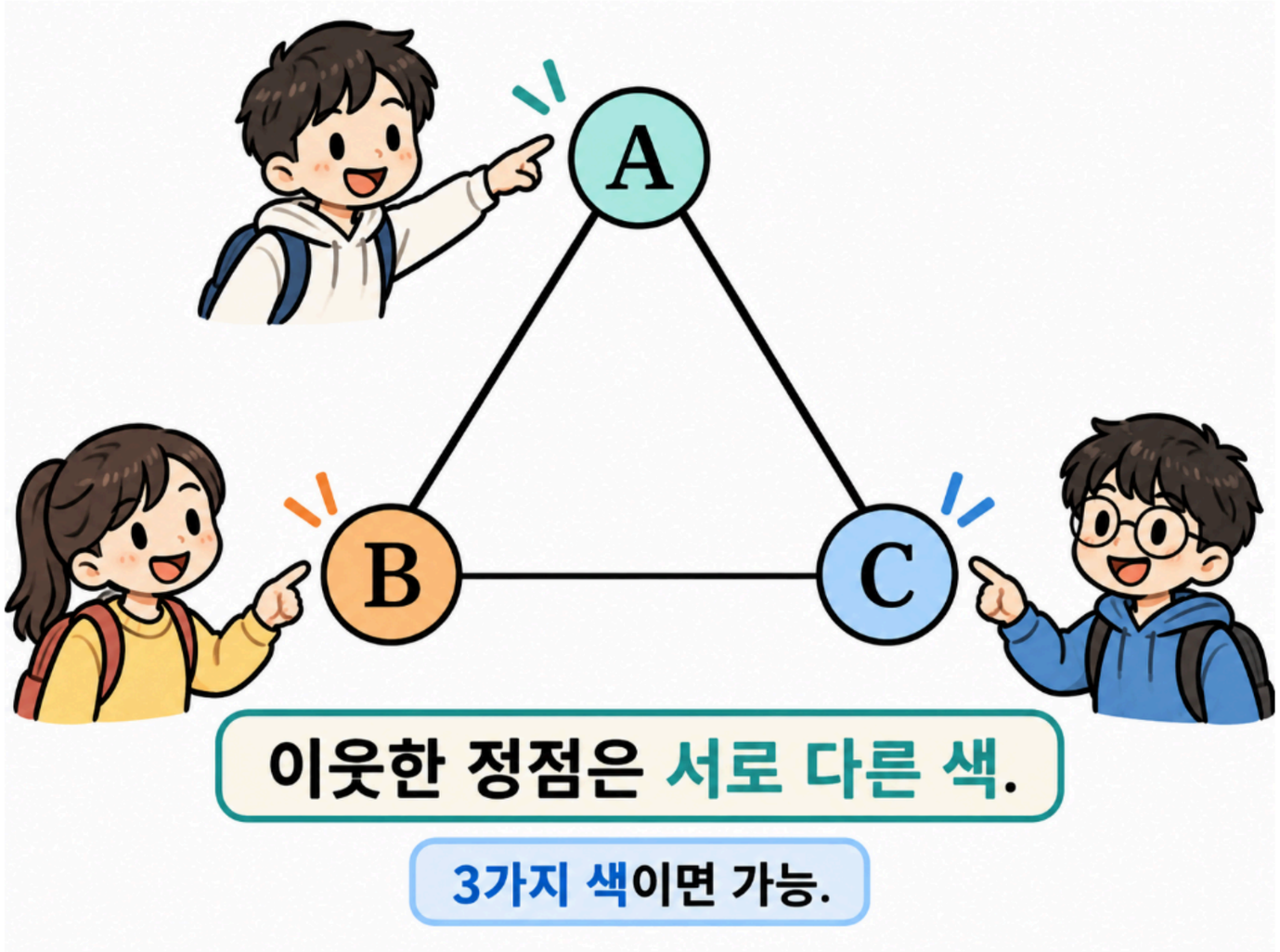
작은 예

세 점 A, B, C가 삼각형으로 연결되어 있습니다.
A를 빨강, B를 파랑으로 칠하면 C에는 다른 색이 필요합니다.

반환할 값

세 색이면 가능하므로 1, 두 색이면 불가능하므로 0을 반환합니다.

7-3 그림으로 보는 예



삼각형의 세 점
서로 모두 이웃입니다.
세 색이면 가능
두 색이면 불가능

7-3 하나씩 계산한 결과

칠할 점	시도할 색	결과
A	빨강	가능
B	빨강	A와 이웃하므로 제외
B	파랑	가능
C	빨강	A와 이웃하므로 제외
C	파랑	B와 이웃하므로 제외
C	초록	세 색을 쓸 수 있으면 가능

한 점의 색을 고를 때 이미 칠한 이웃만 확인하면 됩니다.

연결을 0과 1의 표로 적습니다

삼각형 A,B,C를 코드에서는 0,1,2로 부릅니다.

출발 점	점0	점1	점2
점0	0	1	1
점1	1	0	1
점2	1	1	0

graph[0][2]=1이므로 점0과 점2는 서로 연결되어 있습니다.

7-3 계산 순서

색 하나 시도

현재 점에 1부터 k까지의 색을 넣어 봅니다.

같은 색 이웃 확인

이웃 중 같은 색이 있으면 그 색을 건너뜁니다.

다음 점 또는 되돌리기

다음 점을 칠해 보고 실패하면 지금 색을 0으로 지웁니다.

7-3 변수와 코드의 뜻

코드	뜻
graph[v][u]==1	v와 u가 선으로 연결됨
color[u]==c	이미 칠한 u의 색이 지금 시도하는 색 c와 같음
color[v]=c	현재 점의 색을 c로 선택
color[v]=0	선택을 지우고 다른 색을 시도

인접 행렬은 연결을 적은 표입니다. 행 v·열 u가 1이면 두 점이 이웃입니다.

7-3 함수에 들어오는 값

함수

can_color(graph, k)

입력 조건

점 1~8개, 색 1~3개. 연결 표는 대칭인 0/1 표이며 대각선은 0입니다.

7-3 Python (1/2)

```
def can_color(graph, k):
    n = len(graph)
    color = [0]*n
    def paint(v):
        if v == n:
            return 1
        for c in range(1, k+1):
            allowed = True
            for u in range(v):
                if graph[v][u] == 1 and color[u] == c:
```

다음 쪽에서 이어집니다.

7-3 Python (2/2)

```
                    allowed = False
                    break
            if allowed:
                color[v] = c
                if paint(v+1) == 1:
                    return 1
                color[v] = 0
        return 0
    return paint(0)
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

7-3 C (1/2)

```
static int paint(const int g[][8], int n, int k,
    int v, int color[]) {
    if (v==n) return 1;
    for (int c=1; c<=k; ++c) {
        int ok=1;
        for (int u=0; u<v; ++u)
            if (g[v][u] && color[u]==c) ok=0;
        if (ok) {
            color[v]=c;
            if (paint(g,n,k,v+1,color)) return 1;
```

다음 쪽에서 이어집니다.

7-3 C (2/2)

```
            color[v]=0;
        }
    }
    return 0;
}
int can_color(const int graph[][8], int n, int k) {
    int color[8]={0};
    return paint(graph,n,k,0,color);
}
```

보수적 최악 시간 $O(n^2 \cdot k^n)$, 색 배열·재귀 스택 $O(n)$

7-3 다른 입력도 확인합니다

그래프, 색 수	답	이유
점 하나, 한 색	1	이웃이 없음
선 하나, 한 색	0	양 끝을 다르게 못 칠함
삼각형, 두 색	0	마지막 점의 색이 없음
삼각형, 세 색	1	각 점을 다른 색으로 칠함

인접 행렬은 연결을 적은 표입니다. 행 v·열 u가 1이면 두 점이 이웃입니다.

7-4 N-Queen 배치의 수

구할 것

퀸끼리 같은 행·열·대각선에 놓이지 않는 배치 수를 셉니다.

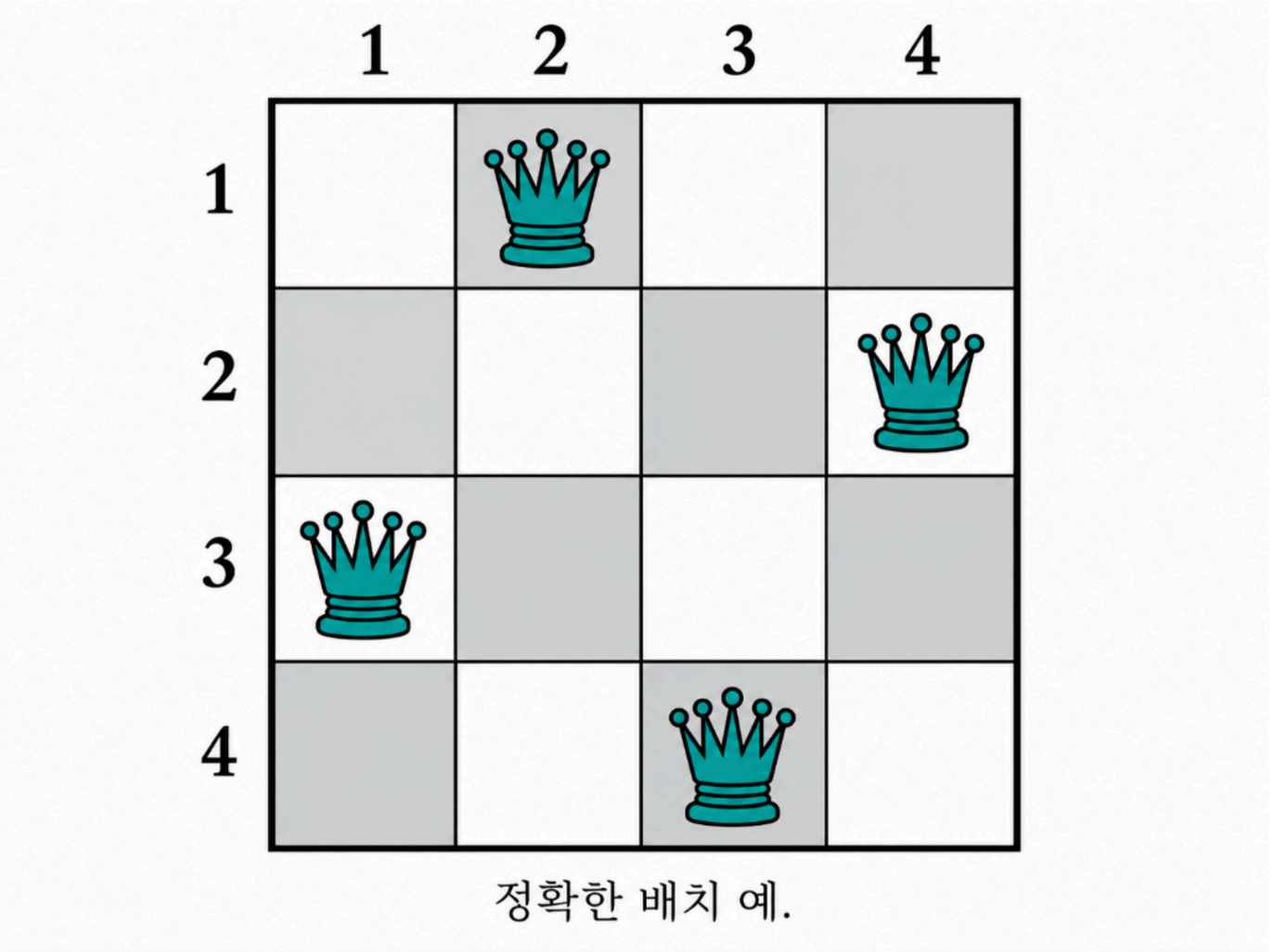
작은 예

4×4 체스판에 퀸 네 개를 놓습니다.
위에서부터 열 번호를 2,4,1,3으로 고르면 서로 공격하지 않습니다.

반환할 값

4×4의 가능한 배치는 총 두 가지입니다. 2를 반환합니다.

7-4 그림으로 보는 예



정확한 배치 예.

위에서부터
2, 4, 1, 3열
같은 열이 없고
같은 대각선도 없습니다.

7-4 하나씩 계산한 결과

놓을 행	열	앞 퀸과 충돌 여부
첫째	2	앞 퀸 없음
둘째	4	열 다름, 대각선 아님
셋째	1	앞의 두 퀸과 충돌 없음
넷째	3	앞의 세 퀸과 충돌 없음

한 행에 하나씩 놓으므로 같은 행 여부는 따로 검사하지 않아도 됩니다.

그림의 열 번호와 코드의 열 번호

그림은 1부터, 배열과 코드는 0부터 셉니다.

그림의 행	그림의 열	코드의 열
1	2	1
2	4	3
3	1	0
4	3	2

같은 배치를 코드의 columns에는 [1,3,0,2]로 저장합니다.

7-4 계산 순서

한 행씩 진행

현재 행에서 퀸을 놓을 열을 하나 고릅니다.

앞 행들과 비교

같은 열이거나 행 차이와 열 차이가 같으면 제외합니다.

모든 행을 채우면

배치 하나를 찾았으므로 1을 더합니다.

7-4 변수와 코드의 뜻

코드	뜻
columns.append(col)	현재 행에서 고른 열을 목록에 추가
col != c	앞 퀸의 열 c와 이번 열 col이 달라야 함
abs(col−c) != row−r	열 차이와 행 차이가 달라야 대각선 충돌 없음
columns.pop()	방금 고른 열을 지우고 다른 열도 시도

예를 들어 (1행,2열)과 (2행,3열)은 행·열 차이가 모두 1이어서 같은 대각선입니다.

7-4 함수에 들어오는 값

함수

nqueen_count(n)

입력 조건

n은 1~8입니다. 회전·대칭 배치도 놓인 칸이 다르면 따로 셉니다.

7-4 Python (1/2)

```
def nqueen_count(n):
    columns = []
    def place(row):
        if row == n:
            return 1
        answer = 0
        for col in range(n):
            allowed = True
            for r in range(row):
                c = columns[r]
```

다음 쪽에서 이어집니다.

7-4 Python (2/2)

```
            if col == c or abs(col-c) == row-r:
                allowed = False
                break
        if allowed:
            columns.append(col)
            answer += place(row+1)
            columns.pop()
    return answer
return place(0)
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

7-4 C (1/2)

```
static int queens(int n, int row, int pos[]) {
    if (row==n) return 1;
    int answer=0;
    for (int col=0; col<n; ++col) {
        int ok=1;
        for (int r=0; r<row; ++r) {
            int d=col-pos[r];
            if (d<0) d=-d;
            if (col==pos[r] || d==row-r) ok=0;
        }
        if (ok) {
```

다음 쪽에서 이어집니다.

7-4 C (2/2)

```
            pos[row]=col;
            answer+=queens(n,row+1,pos);
        }
    }
    return answer;
}
int nqueen_count(int n) {
    int pos[8]={0};
    return queens(n,0,pos);
}
```

열 중복을 배제하며 탐색. 충돌 확인을 포함한 보수적 시간 $O(n^2 \cdot n!)$, 공간 $O(n)$

7-4 다른 입력도 확인합니다

n	답	이유
1	1	한 칸에 하나
2	0	서로 공격하게 됨
3	0	세 퀸을 모두 놓을 수 없음
4	2	뒤집은 배치도 따로 셈

예를 들어 (1행,2열)과 (2행,3열)은 행·열 차이가 모두 1이어서 같은 대각선입니다.

7장의 계산을 한 문장으로

문제	구하는 값
7-1	서로 다른 카드 중 몇 장을 골라 줄 세우는 경우의 수를 셉니다.
7-2	몇 개의 수를 골라 합이 목표와 같아지는 경우를 셉니다.
7-3	선으로 연결된 두 점의 색을 다르게 칠할 수 있는지 확인합니다.
7-4	퀸끼리 같은 행·열·대각선에 놓이지 않는 배치 수를 셉니다.

하나 골라 보고, 안 맞으면 돌아와 다른 것을 골라 봅니다.